

Security in Software Applications

Language-Based Security – Overview of types



SAPIENZA
UNIVERSITÀ DI ROMA

Daniele Friolo

friolo@di.uniroma1.it

<https://danielefriolo.github.io>

Language-based Security

- Objective
 - Design security into the language
 - Compiler rules out insecure programs
 - Compiler does not run the program (no testing)
- What is insecure?
 - Buffer overflows
 - Information-flow leaks
 - Violations of access rights
 - ...
- One approach is based on *types*

TYPES: Overview

A *type* is a *specification* of data or code in a program

Examples from C:

- Basic types

- `int, char, float, double, void`
- `int x;` -- variable `x` will store an integer

- Function types

- `int -> double, ...`
- `int factorial(int);` --- `factorial` is a function that takes an integer as an argument and returns an integer (type `int -> int`)

TYPES: Overview

■ Structures types

- `struct pair {int x; int y;};`

■ Array types

- `int[n], char[n][m], ...`
- `int arr[8];` --- variable `arr` stores sequence of 8 integers

■ Pointer types

- `int*, int**, ...`
- `int* p;` --- variable `p` stores the address of a variable that stores an integer
- `int** p;` --- variable `p` stores the address of a variable that stores the address of a variable that stores an integer

Type-correct program

```
int readint();           // readint: void -> int
void writeint(int);      // writeint: int -> void
int factorial(int n) {   // factorial: int -> int
    int f = 1;           // f: int
    int c = 1;           // c: int
    while (c <= n) {     // c <= n: boolean
        f = f * c;       // *: (int,int) -> int
        c++;            // ++: int -> int
    }                   // f: int
    return f;
}

void main() {           // main: void -> void
    int v;              // v: int
    v = readint();       // readint(): int
    writeint(factorial(v)); // factorial(v): int
}
```

Type-correct program

```
int readint();
void writeint(int);
int factorial(int n) {
    int f = 1; int c = 1;
    while (c <= n) {
        f = f * c;
        c++;
    }
    return f;
}
void main() {
    string s;
    s = readint();
    writeint(factorial(s));
}
```

```
// readint: void -> int
// writeint: int -> void
// factorial: int -> int
// f: int
// c: int
// c <= n: boolean
// *: (int,int) -> int
// ++: int -> int

// f: int

// main: void -> void
// s: string
// readint(): int -- error
// factorial(s): -- no type
```

Why types? Compilation

Types are *necessary* to compile source code

- What is the binary representation of variables?

- `int x; --- x is 4 bytes`
- `char x; --- x is 1 byte`
- `int arr[8]; --- arr is 32 bytes`

- How to compute in assembly?

- `int x; int y; x + y → add r1, r2`
- `float x; float y; x + y → fadd r1, r2`
- Difference is based on types

Why types? Reject "bad" programs

Type checking: A compiler can *ensure that data and code are used only as specified by types* without running the program

Types can be used to reject at compile time programs that:

- Use strings as integers
- Use integers as pointers
- Cause null-pointer exceptions
- Cause array overflows
- Leak secret information

Type safety definitions

- A *program* is called *safe* if it is not stuck and has not crashed
- A *language* is called *type-safe* if well-typed programs always remain safe
- Presence of types does not imply type-safety
 - e.g., C has types but is not type-safe

Buffer overflow with scanf in C

```
void readstring(char str[])
{
    scanf("%s", str);
}

void main()
{
    char buf[8];
    readstring(buf);
    ...
}
```

This program is well-typed, but can crash with input longer than 7 bytes

C is not type-safe

Buffer overflow with scanf in C – What is wrong?

```
void readstring(char str[])
{
    scanf("%s", str);
}

void main()
{
    char buf[8];
    readstring(buf);
    ...
}
```

Unbounded length of the array
(type char[])

Buf has length 8
(type char[8])

`char[] != char[8]`

This program should be rejected! However, C allows unsafe *type-casting* and accepts the program

Unsafe type-casting in C

Type-casting: convert the type of a variable to another

- The C compiler will convert types (e.g., from `char[8]` to `char[]`)
- Some conversions violate type specifications
- This makes C type-unsafe

Safe and Unsafe type cast in C

Type-cast

- `char* x; int* y = (int*) x`
- `int* x; void* y = (void*) x`
- `void* x; int* y = (int*) x`
- `int x; char* y = (char*) x`
- `char* x; int y = (int) x`
- `int x; float y = (float) x`
- `float x; int y = (int) x`

Safe in C?

- No
- Yes
- No
- No
- Yes
- Yes
- Yes

Absent, weak and strong typing

Untyped languages do not have types

- e.g., Bash, Perl, Ruby, ...
- Usually interpreted; difficult to compile without types
- Program safety is programmer's responsibility: programs difficult to debug

Weakly typed languages use types only for compilation; no type-safety

- e.g., C, C++, etc.
- Often allow unsafe casts, e.g., `char[8]` to `char[]` and `int` to `char*`
- Program safety is programmer's responsibility: buffer overflows and segmentation faults are common in programs

Strongly typed languages use types for compilation and guarantee type-safety

- e.g., BASIC, Pascal, Cyclone, Haskell, SML, Java, etc.
- No unsafe casts, e.g., an integer cannot be cast to a pointer, an array of length 8 is not an unbounded array, etc.
- Safety is guaranteed but believed unsuitable for some low-level programs (debatable)

Summary of types

- Types are specifications of data and code
- Compiler may check well-typedness without executing the program
- Existence of type specifications may imply program safety (type-safety)
- Not all languages with types are type-safe
 - e.g., C is not type-safe

Now we take a simple type-safe language and understand type-safety more formally

Mini-C: A strongly typed language

No functions
No structures
No pointers
No casts

Types $T ::= \text{int} \mid \text{bool}$

Variables x, y

Declarations $\Delta ::= x_1 : T_1; \dots; x_n : T_n$

Integers $n ::= \dots \mid -1 \mid 0 \mid 1 \mid \dots$

Expressions $e ::= x \mid \text{true} \mid \text{false} \mid n \mid e_1 + e_2 \mid e_1 * e_2 \mid e_1 \leq e_2 \mid e_1 == e_2$

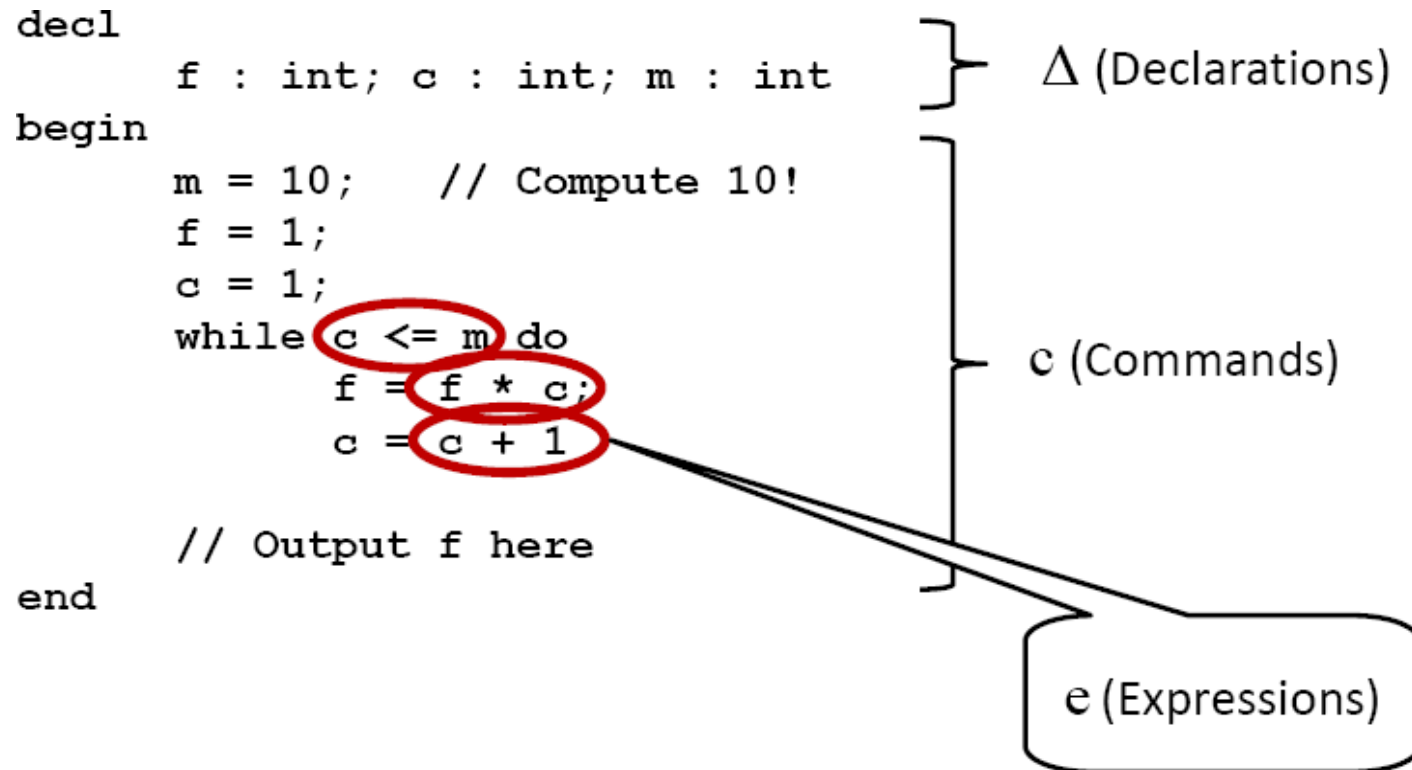
Commands $c ::= \text{noop} \mid x = e \mid c_1; c_2 \mid \text{if } e \text{ then } c_1 \text{ else } c_2 \mid \text{while } e \text{ do } c$

Programs $P ::= \text{decl } \Delta \text{ begin } c \text{ end}$

Values $v ::= \text{true} \mid \text{false} \mid n$

Stores $\sigma ::= x_1 \mapsto v_1, \dots, x_n \mapsto v_n$

Mini-C Factorial program



Type safety through semantics

- Semantics define safe program computation
- Unsafe programs are those that cannot compute
- Type-safety: A correctly typed program is never stuck
- Game-plan:
 - Describe semantics for Mini-C
 - Describe typing for Mini-C
 - Prove that typing $\Rightarrow$ Not stuck (*Type-safety*)

Semantic of expressions

Expressions $e ::= x \mid \text{true} \mid \text{false} \mid n \mid e_1 + e_2 \mid e_1 * e_2 \mid e_1 \leq e_2 \mid e_1 == e_2$

Values $v ::= \text{true} \mid \text{false} \mid n$

- Expressions compute by evaluation to values
- Formalized by a reduction relation:

$$\sigma \triangleright e \hookrightarrow e'$$

Stores $\sigma ::= x_1 \mapsto v_1, \dots, x_n \mapsto v_n$

Semantic of expressions

- To evaluate a variable, read its value from memory

$$\frac{(x \mapsto v) \in \sigma}{\sigma \triangleright x \hookrightarrow v}$$

- To evaluate $(e_1 + e_2)$, first evaluate e_1 , then evaluate e_2 , and then **add** them

$$\frac{\sigma \triangleright e_1 \hookrightarrow e'_1}{\sigma \triangleright e_1 + e_2 \hookrightarrow e'_1 + e_2}$$

$$\frac{\sigma \triangleright e_2 \hookrightarrow e'_2}{\sigma \triangleright v_1 + e_2 \hookrightarrow v_1 + e'_2}$$

$$\frac{\text{add}(n_1, n_2) = n}{\sigma \triangleright n_1 + n_2 \hookrightarrow n}$$

Semantic of expressions

■ Similar rules for other expressions

Important: No rules to evaluate unsafe expressions like $(\text{true} * 7)$

$$\frac{\sigma \triangleright e_1 \hookrightarrow e'_1}{\sigma \triangleright e_1 * e_2 \hookrightarrow e'_1 * e_2}$$

$$\frac{\sigma \triangleright e_2 \hookrightarrow e'_2}{\sigma \triangleright v_1 * e_2 \hookrightarrow v_1 * e'_2}$$

$$\frac{\text{mult}(n_1, n_2) = n}{\sigma \triangleright n_1 * n_2 \hookrightarrow n}$$

$$\frac{\sigma \triangleright e_1 \hookrightarrow e'_1}{\sigma \triangleright e_1 \leq e_2 \hookrightarrow e'_1 \leq e_2}$$

$$\frac{\sigma \triangleright e_2 \hookrightarrow e'_2}{\sigma \triangleright v_1 \leq e_2 \hookrightarrow v_1 \leq e'_2}$$

$$\frac{\text{leq}(n_1, n_2) = b}{\sigma \triangleright n_1 \leq n_2 \hookrightarrow b}$$

$$\frac{\sigma \triangleright e_1 \hookrightarrow e'_1}{\sigma \triangleright e_1 == e_2 \hookrightarrow e'_1 == e_2}$$

$$\frac{\sigma \triangleright e_2 \hookrightarrow e'_2}{\sigma \triangleright v_1 == e_2 \hookrightarrow v_1 == e'_2}$$

$$\frac{\text{eq}(n_1, n_2) = b}{\sigma \triangleright n_1 == n_2 \hookrightarrow b}$$

Semantic of commands

Commands $c ::= \text{noop} \mid x = e \mid c_1; c_2 \mid \text{if } e \text{ then } c_1 \text{ else } c_2 \mid \text{while } e \text{ do } c$

- Commands compute to noop, but change memory
- Formalized by a reduction relation

$$\sigma ; c \longrightarrow \sigma' ; c'$$

- Starting from memory σ , command c reduces to command c' and updates memory to σ'

Semantic of commands

- To evaluate $(x = e)$, first evaluate expression e to a value v . Then update memory by putting the value v in x

$$\boxed{\frac{\sigma \triangleright e \hookrightarrow^* v}{\sigma ; (x = e) \longrightarrow \sigma[x \mapsto v] ; \text{noop}}}$$

- To evaluate $(c_1 ; c_2)$, first evaluate c_1 , then evaluate c_2

$$\frac{\sigma ; c_1 \longrightarrow \sigma' ; c'_1}{\sigma ; (c_1 ; c_2) \longrightarrow \sigma' ; (c'_1 ; c_2)}$$

$$\frac{}{\sigma ; (\text{noop} ; c_2) \longrightarrow \sigma ; c_2}$$

Semantic of commands

- Similar rules for other commands

$$\frac{\sigma \triangleright e \hookrightarrow^* \text{true}}{\sigma ; (\text{if } e \text{ then } c_1 \text{ else } c_2) \longrightarrow \sigma ; c_1}$$

$$\frac{\sigma \triangleright e \hookrightarrow^* \text{false}}{\sigma ; (\text{if } e \text{ then } c_1 \text{ else } c_2) \longrightarrow \sigma ; c_2}$$

$$\frac{\sigma \triangleright e \hookrightarrow^* \text{true}}{\sigma ; (\text{while } e \text{ do } c) \longrightarrow \sigma ; (c ; (\text{while } e \text{ do } c))}$$

$$\frac{\sigma \triangleright e \hookrightarrow^* \text{false}}{\sigma ; (\text{while } e \text{ do } c) \longrightarrow \sigma ; \text{noop}}$$

Important: No rules to evaluate unsafe expressions like
(if 7 then c else c')

Summary of semantics

- Semantics are rules for evaluating programs safely
- Programs that cannot evaluate are unsafe by definition
- Two types of reductions, for expressions and commands

$$\sigma \triangleright e \hookrightarrow e'$$

$$\sigma ; c \longrightarrow \sigma' ; c'$$

Type checking

Declarations Δ ::= $x_1 : T_1; \dots; x_n : T_n$

- Typing for expressions

$$\Delta \vdash e : T$$

- Typing for commands

$$\Delta \vdash c$$

Important: A compiler can check efficiently whether a program is well typed or not

Typing for expressions

- To type check a variable, find its declaration

$$\boxed{\frac{(x : T) \in \Delta}{\Delta \vdash x : T}}$$

- Both true and false have type bool

$$\overline{\Delta \vdash \text{true} : \text{bool}}$$

$$\overline{\Delta \vdash \text{false} : \text{bool}}$$

- A number has type int

$$\boxed{\overline{\Delta \vdash n : \text{int}}}$$

Typing for expressions

- $(e_1 + e_2)$ has type int if both e_1 and e_2 have type int

$$\frac{\Delta \vdash e_1 : \text{int} \quad \Delta \vdash e_2 : \text{int}}{\Delta \vdash e_1 + e_2 : \text{int}}$$

- Other expressions are typed similarly

$$\frac{\Delta \vdash e_1 : \text{int} \quad \Delta \vdash e_2 : \text{int}}{\Delta \vdash e_1 * e_2 : \text{int}} \quad \frac{\Delta \vdash e_1 : \text{int} \quad \Delta \vdash e_2 : \text{int}}{\Delta \vdash e_1 \leq e_2 : \text{bool}}$$
$$\frac{\Delta \vdash e_1 : \text{int} \quad \Delta \vdash e_2 : \text{int}}{\Delta \vdash e_1 == e_2 : \text{bool}}$$

Typing for commands

$$\Delta \vdash c$$

$\frac{}{\Delta \vdash \text{noop}}$	$\frac{(x : T) \in \Delta \quad \Delta \vdash e : T}{\Delta \vdash x = e}$	$\frac{\Delta \vdash c_1 \quad \Delta \vdash c_2}{\Delta \vdash c_1 ; c_2}$
$\frac{\Delta \vdash e : \text{bool} \quad \Delta \vdash c_1 \quad \Delta \vdash c_2}{\Delta \vdash \text{if } e \text{ then } c_1 \text{ else } c_2}$		$\frac{\Delta \vdash e : \text{bool} \quad \Delta \vdash c}{\Delta \vdash \text{while } e \text{ do } c}$

Typing: example

Note: $\Delta = f : \text{int}; c : \text{int}; m : \text{int}$

$$\mathcal{D}_1 = \frac{\frac{(f : \text{int}) \in \Delta}{\Delta \vdash f : \text{int}} \quad \frac{\frac{(c : \text{int}) \in \Delta}{\Delta \vdash c : \text{int}}}{\Delta \vdash f * c : \text{int}}}{\Delta \vdash f = f * c}$$

$$\mathcal{D}_2 = \frac{\frac{(c : \text{int}) \in \Delta}{\Delta \vdash c : \text{int}} \quad \frac{\frac{(c : \text{int}) \in \Delta}{\Delta \vdash c + 1 : \text{int}}}{\Delta \vdash c = c + 1}}$$

$$\mathcal{D} = \frac{\frac{\frac{(c : \text{int}) \in \Delta}{\Delta \vdash c : \text{int}} \quad \frac{(m : \text{int}) \in \Delta}{\Delta \vdash m : \text{int}}}{\Delta \vdash c \leq m : \text{bool}} \quad \frac{\mathcal{D}_1 \quad \mathcal{D}_2}{\Delta \vdash f = f * c; c = c + 1}}{\Delta \vdash \text{while } (c \leq m) \text{ do } (f = f * c; c = c + 1)}$$

Typing for memory

- Typing for memory ensures that variables in memory hold values of expected type

$$\Delta \vdash \sigma$$

if and only if

$(x : \text{int}) \in \Delta$ implies $(x \mapsto n) \in \sigma$ for some n

$(x : \text{bool}) \in \Delta$ implies $(x \mapsto \text{true}) \in \sigma$ or $(x \mapsto \text{false}) \in \sigma$

Type safety

Suppose

- c is a well-typed command
 - σ is a well-typed memory
 - $\sigma; c \rightarrow^* \sigma'; c'$
-
- Then c' is not stuck, i.e., it is noop, or it can reduce further

Type safety formally

Theorem (Safety for commands). *Suppose the following holds:*

1. $\Delta \vdash c$
2. $\Delta \vdash \sigma$
3. $\sigma; c \rightarrow^* \sigma'; c'$

Then either $c' = \text{noop}$ or there are σ'' and c'' such that $\sigma'; c' \rightarrow^ \sigma''; c''$*

Type safety

- Why is this theorem “type-safety”?
- Theorem says that a well-typed program will either terminate or keep reducing
- By definition, such a program is safe
- Therefore, a typed program always remains safe

Summary: types and type-safety

- *Types* specify expected program behavior
- Semantics define what safe behavior is
- *Type-safety*: Typing $\Rightarrow$ always follow semantics
 $\Rightarrow$ always safe behavior
- *Reminder*: Not all typed languages are type-safe

What else types can do

- Prevent null-pointer dereferencing and dangling pointers (e.g., Cyclone)
- Enforce array bounds (e.g., DML)
- Prevent information leaks (e.g., Jif, Sif)
- Enforce access control (e.g., PCML5, Aura, PCAL)
- Ensure correctness of protocols (e.g., F7)